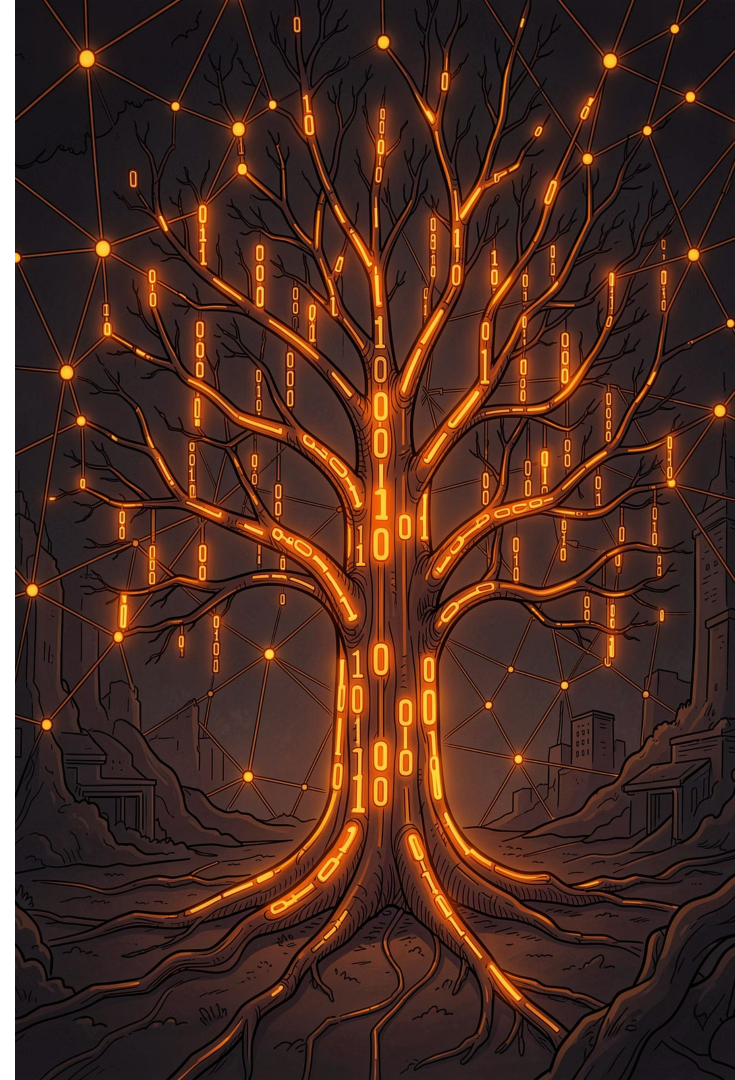


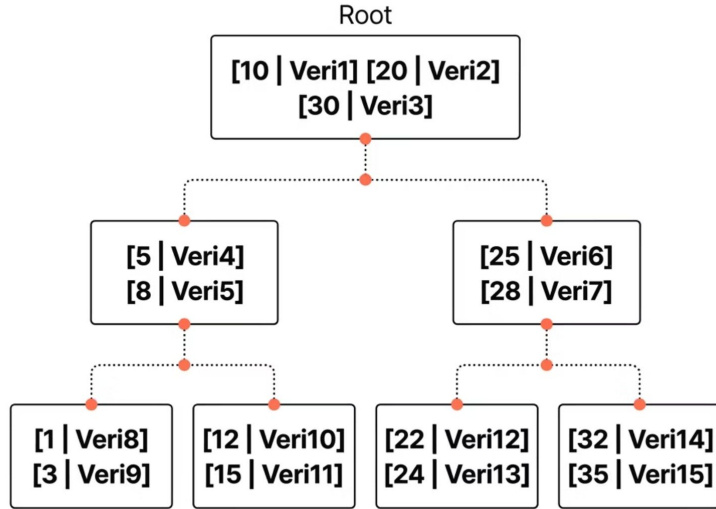
Dijital Dünyanın Kökleri: B ve B+ Ağaçlarının Modern Sistemlerdeki Rolü

Veri Tabanlarından Büyük Veriye, Bilgiyi Organize Etme Sanatı

NIHAT BURAK ERASLAN – İNÖNÜ ÜNİVERSİTESİ YAZILIM MÜHENDİSLİĞİ



Teorik Temeller: B ve B+ Ağaçları Nedir?



B-Tree

Kendi kendini dengeleyen bir yapı. Her düğümde hem **indeks anahtarları** hem de **gerçek veri** birlikte bulunur. Arama, ekleme ve silme işlemleri $O(\log n)$ zaman karmaşıklığıyla gerçekleşir.

B+ Tree

İç düğümlerde yalnızca **indeks** bulunur; tüm gerçek veriler en alttaki **yaprak düğümlerde** saklanır. Yapraklar birbirine **bağlı liste** ile zincirlenmiştir, bu da ardışık taramayı son derece verimli kılar.

Günlük Hayattan Benzetme: Akıllı Hastane Arşivi

B-Tree Arşivi

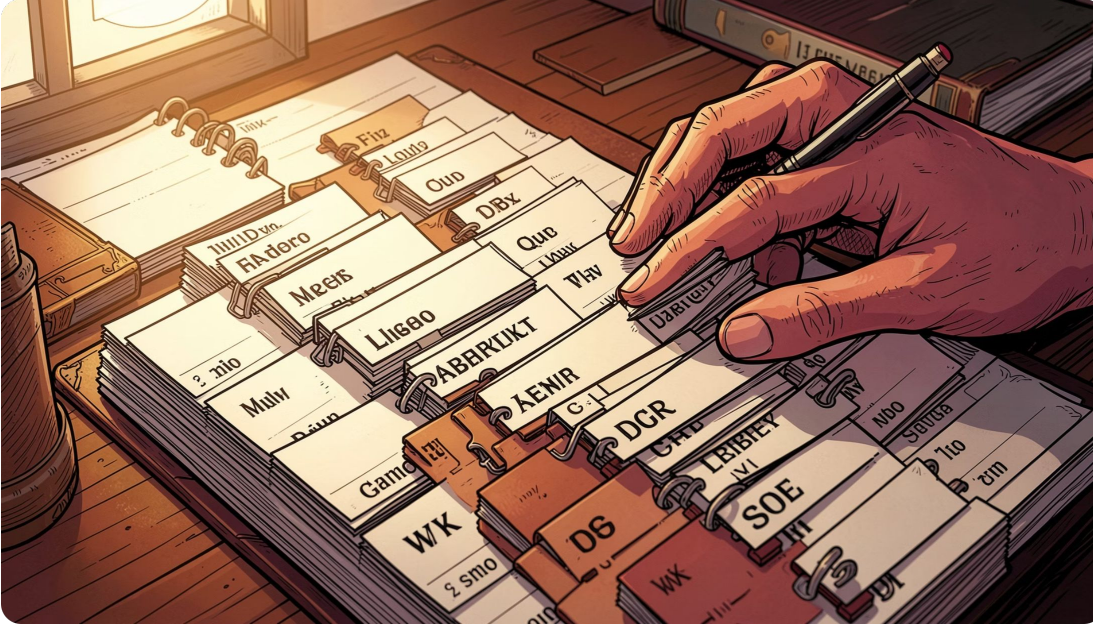
Her dolabın üzerinde harf etiketi vardır ve içinde hem **hasta dosyası** hem de alt çekmeceye yönlendiren **kılavuz kartlar** bulunur. Her adımda hem veriye hem yönlendirmeye ulaşılır.

B+ Tree Arşivi

Üst katlarda yalnızca **yönlendirme kartları** bulunur. Tüm hasta dosyaları **bodrum katta** yan yana, alfabetik sırayla ve birbirine zincirli şekilde dizilidir. Yönlendirme ile veri katmanı tamamen ayrılmıştır.



Bu Analojinin Somut Faydası: Aralık Sorguları



Nokta Atışı Sorgu

Tek bir hastayı ararken iki yapı da **3-4 adımda** hedefe ulaşır. Performans farkı bu noktada minimaldir.

Aralık Sorgusu (Range Query)

"K ile M harfi arasındaki dosyaları getir" dendiğinde, **B+ Tree** bodrum kata iner ve zincirli yapı sayesinde tüm dosyaları **tek seferde, düz bir çizgide** toplar. B-Tree ise her düğümü ayrı ayrı ziyaret etmek zorunda kalır.



Aralık sorgularında B+ Tree'nin bağlı liste yapısı, disk erişim sayısını dramatik biçimde azaltır.

Modern Sistemlerde Neden Tercih Edilir?



Disk I/O Optimizasyonu

Disk blok boyutuna tam oturan düğümler sayesinde **tek okumada** yüzlerce indeks RAM'e çekilir. Her disk erişimi pahalıdır; B ağaçları bu maliyeti minimize eder.



Logaritmik Zaman — $O(\log n)$

Milyarlarca kayıta bile ağaç derinliği **son derece sığ** kalır. 1 milyar öge için derinlik yalnızca ~20 katmandır.



Önbellek (Cache) Verimliliği

Kök ve üst düğümler **RAM'de sabit** tutulur. Sık erişilen yol her seferinde disk yerine bellekten okunur, bu da hızı katlar.

Veri Tabanı Motorlarındaki Kullanımı

MySQL — InnoDB



Tabloların bizzat kendisi bir **B+ Tree**'dir (Clustered Index). Birincil anahtar üzerinden veri doğrudan bu ağaçta saklanır; ikincil indeksler de ayrı B+ Tree yapar.

PostgreSQL

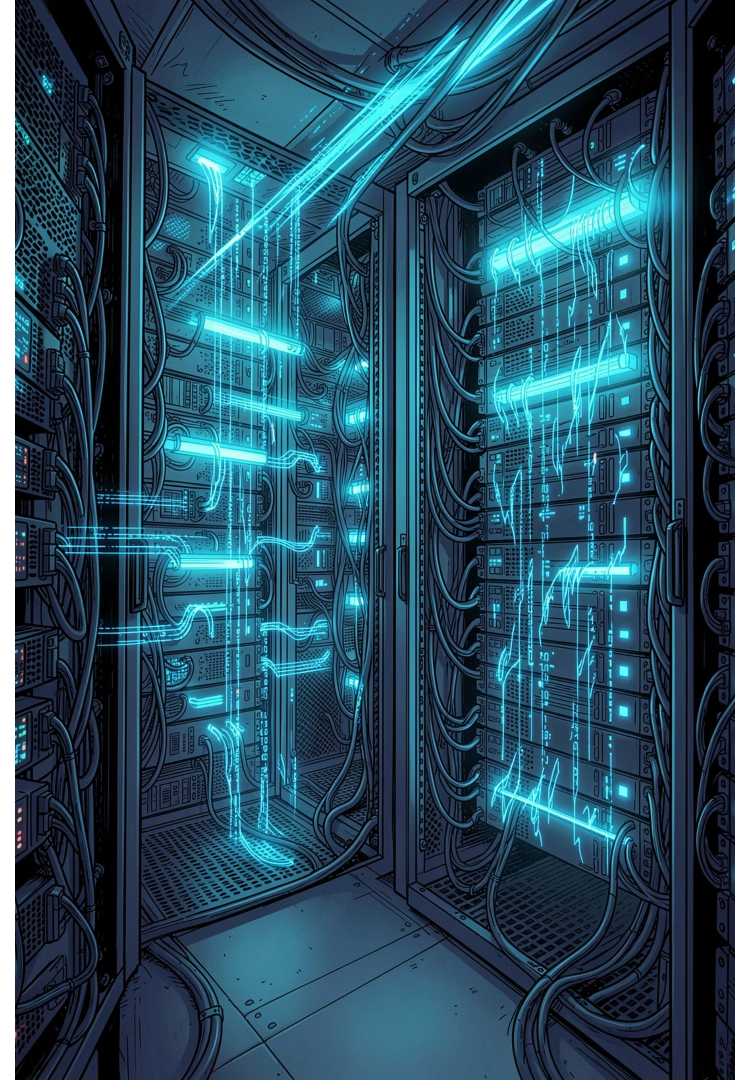


Yüksek eşzamanlılığa (concurrency) izin veren **gelişmiş B-Tree** yapısı kullanır. MVCC ile birlikte çalışarak okuma-yazma çakışmalarını minimize eder.

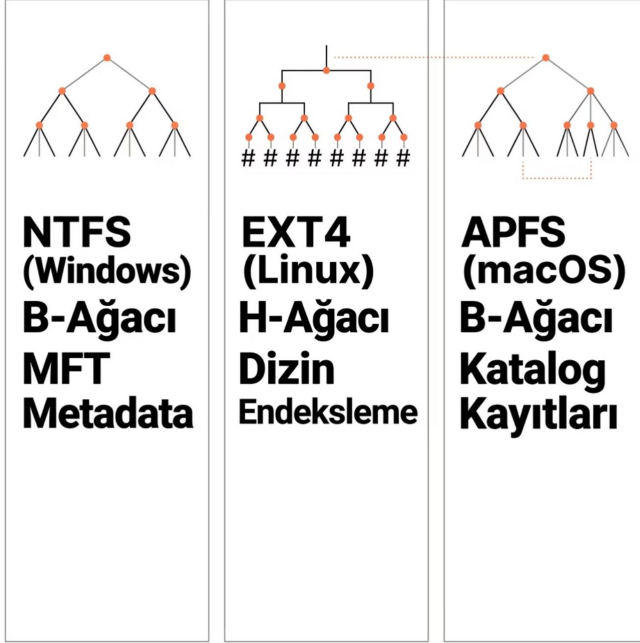
MongoDB — WiredTiger



NoSQL olmasına rağmen indekslemede **B-Tree** kullanır. Doküman tabanlı yapı ile ağaç indeksinin birleşimi, esneklik ve performansı aynı anda sunar.



Dosya Sistemlerindeki Yeri



İşletim Sistemleri ve Ağaç Yapıları

Klasör dizinlerini ve metadata'ları yönetmek için tüm modern işletim sistemleri ağaç yapılarına güvenir. Milyonlarca dosyanın bulunduğu devasa hiyerarşilerde **hızlı erişim** ve **denge** şarttır.

Windows — NTFS

Master File Table (MFT) için B-Tree türevi yapı kullanır.

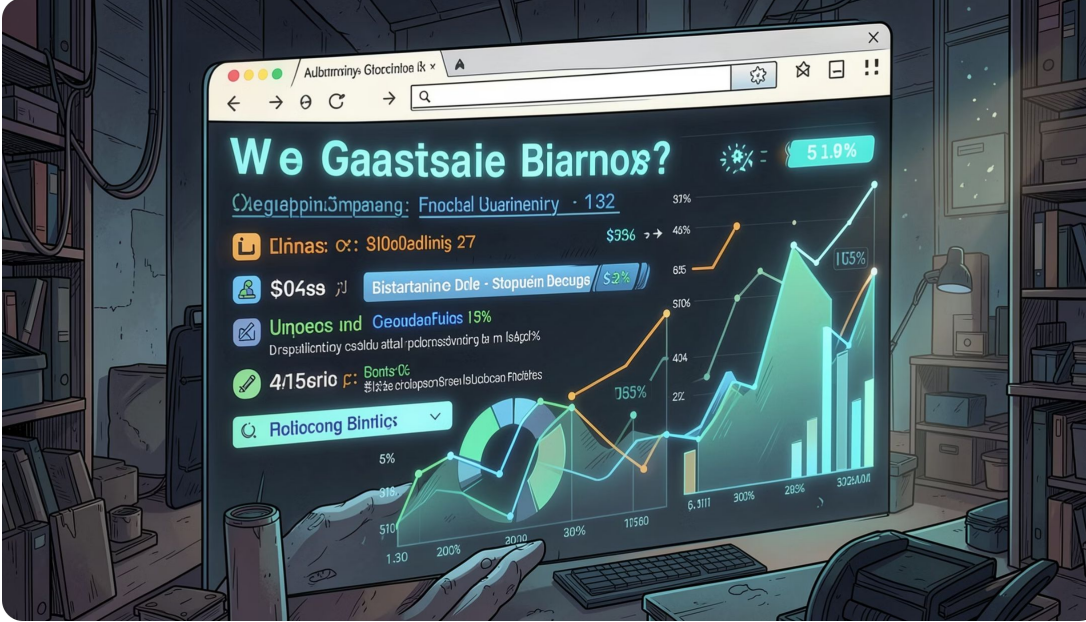
Linux — EXT4

Dizin endeksleme için **H-Tree** (B-Tree varyantı) kullanır.

macOS — APFS

Katalog kayıtları ve metadata için B-Tree yapısını benimser.

Arama Motorları ve Büyük Veri



Arama Motorlarında: Ters İndeks

Arama motorları **Inverted Index** yapısında terim sözlüklerini (Term Dictionary) hızlı taramak için arka planda B-Tree kullanır. Milyarlarca web sayfası arasında milisaniyeler içinde sonuç döndürmek bu sayede mümkün olur.

Büyük Veride: Anlık Analitik

Real-time analytics sistemleri, veriyi **kilitlemeden** (lock-free) raporlama yapabilir. B+ Tree'nin yaprak zinciri, ardışık veri okumalarını son derece verimli kılar ve büyük veri kümelerinde bile anlık sorgulara olanak tanır.

Akademik Değerlendirme I: Neden Bu Kadar Önemli?

B ve B+ ağaçları, yazılım ile donanım arasındaki barış antlaşmasıdır.

İşlemci hızları her yıl katlanarak artarken, mekanik disk ve hatta SSD okuma hızları aynı oranda ilerleyememiştir. Bu **darboğaz** (latency gap), B ağaçlarının disk blok bazlı okuma stratejisiyle çözülmüştür. Tek bir disk erişiminde yüzlerce anahtarın RAM'e çekilmesi, bu yapının neden 50 yıldır vazgeçilmez olduğunu açıklar.

~20

Maximum Derinlik

1 milyar kayıtta ağaç yüksekliği

50+

Yıl Aktif Kullanım

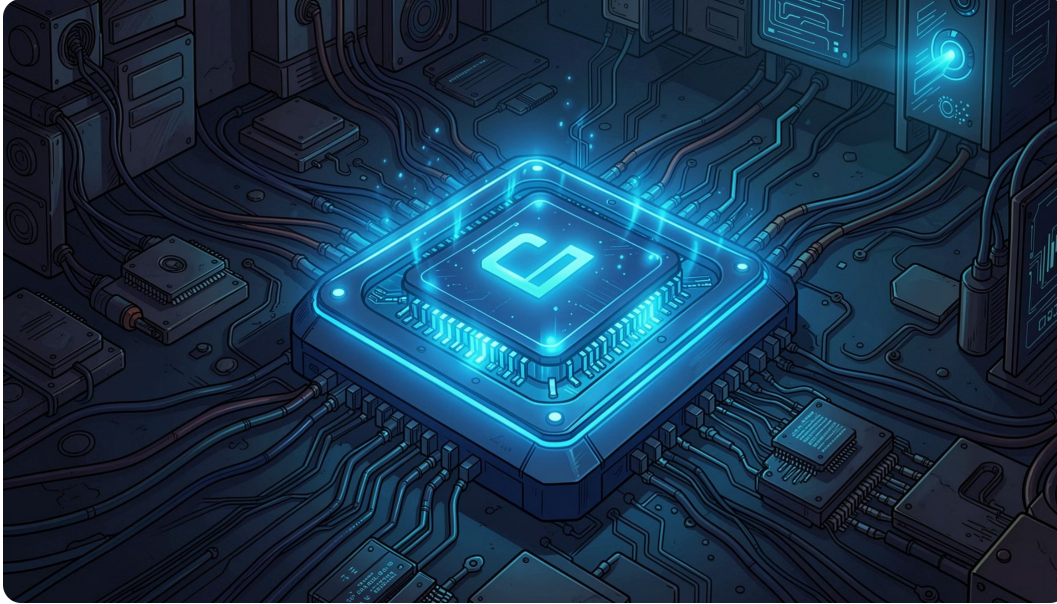
1972'den beri üretim sistemlerinde

100x

I/O Tasarrufu

Naif aramaya kıyasla disk erişim azaltımı

Akademik Değerlendirme II: Gelecekte Kullanılacak mı?



Kesinlikle Evet — Ve İşte Nedeni

Donanımlar **RAM tabanlı** (in-memory) hale gelse veya **kuantum bilgisayarlar** yaygınlaşsa bile, veriyi hiyerarşik olarak **logaritmik bölme mantığı** evrensel bir doğru olarak kalacaktır.



Yapay Zeka Entegrasyonu

Öğrenen indeks yapıları (learned indexes) B-Tree mantığını optimize edecek, ancak temeli değiştirmeyecek.



Evrensel Algoritma

Logaritmik bölme, fiziksel donanımdan bağımsız matematiksel bir zorunluluktur.



Adaptif Gelecek

Yapı evrimleşecek, ancak kök prensip — dengeli, hiyerarşik, logaritmik — sabit kalacak.